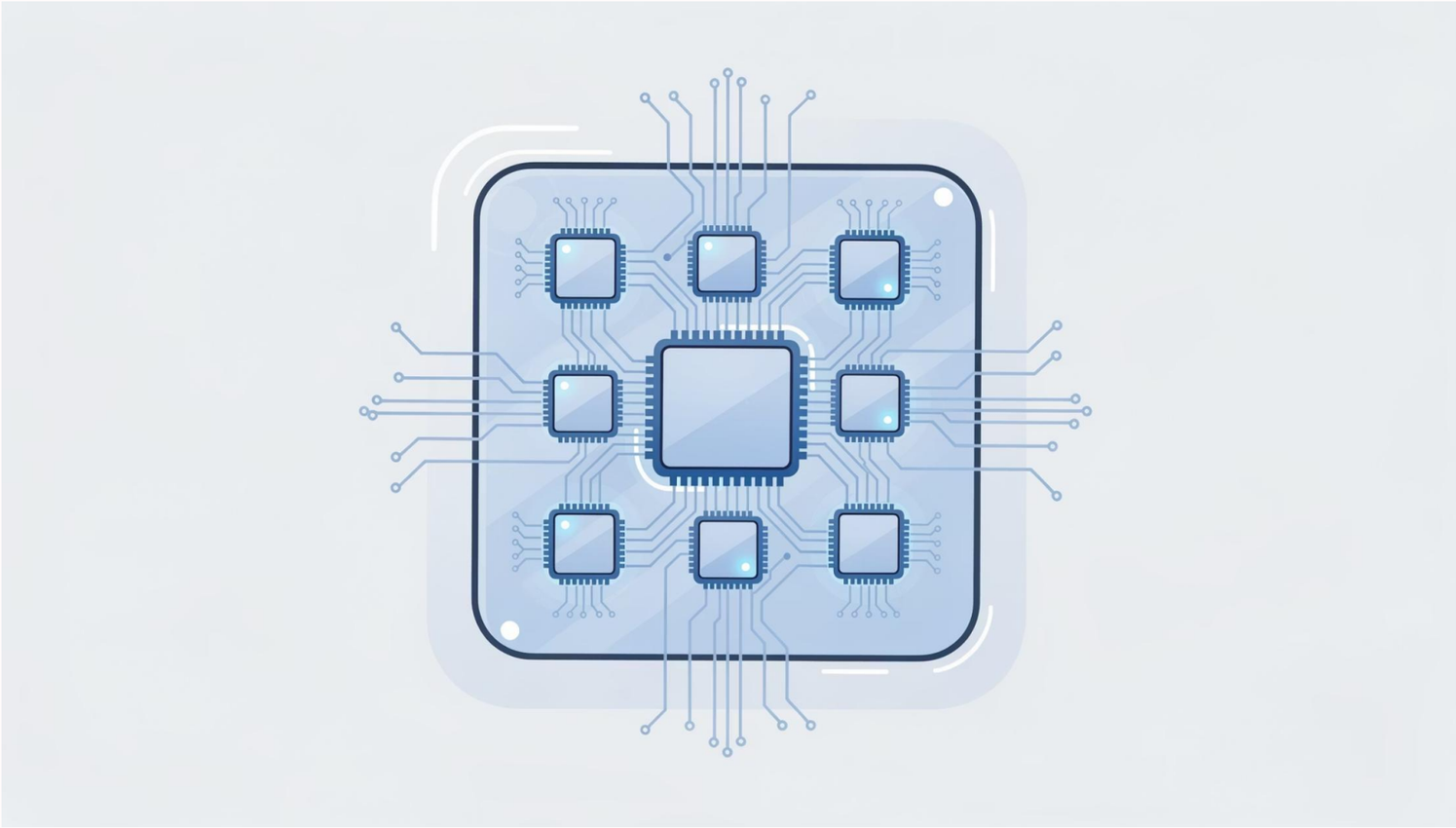
CS 331 Tutorial #6 & #7 Concurrent Programming in Java

An exploration of the fundamental concepts that underpin concurrent programming in Java, revealing why mastering concurrency is essential for modern software development.

FOUNDATION

Why Concurrency Matters



Today's applications face unprecedented demands for responsiveness, scalability, and raw performance. The era of single-core processors improving through clock speed alone has ended.

Multi-core processors have transformed concurrency from an advanced optimisation technique into a fundamental requirement. Applications that cannot leverage multiple threads simply cannot compete.

Concurrency enables multiple tasks to progress simultaneously, dramatically improving throughput and user experience.

The Challenge of Concurrency

Difficult to Write Correctly

Concurrent programs require fundamentally different thinking patterns than sequential code. Even experienced developers struggle with the complexity.

Intermittent Bugs

Concurrency bugs are notoriously elusive—they appear inconsistently, only under specific timing conditions, making them extraordinarily difficult to reproduce and debug.

Sequential Reasoning Fails

Traditional debugging approaches and mental models break down. The assumptions we rely on in sequential programming simply don't hold in concurrent contexts.

The stakes are real. Concurrency bugs have caused catastrophic failures across industries, from medical devices to financial systems.

⚠ CRITICAL HAZARDS

Key Concurrency Hazards

1

Race Conditions

Multiple threads access shared data without proper synchronisation, leading to unpredictable outcomes. The result depends on the precise timing of thread execution—a nightmare for debugging.

2

Deadlocks

Threads become permanently blocked, each waiting for resources held by others in a circular dependency. The application freezes, unable to make any progress.

3

Starvation & Livelocks

Threads continuously attempt to proceed but fail to make meaningful progress. In starvation, threads never get CPU time; in livelocks, they're active but accomplishing nothing.

Benefits of Mastering Concurrency

Improved Throughput

Dramatically increase application performance by executing multiple operations simultaneously, making full use of available CPU cores.

Responsive Interfaces

Keep user interfaces fluid and responsive even during intensive operations by offloading work to background threads.

Hardware Utilisation

Fully leverage modern multi-core processors—code that doesn't use concurrency leaves performance on the table.

Java's Foundation: Built for Concurrency

Java was designed from the ground up with concurrent programming in mind — making it one of the most robust platforms for building multi-threaded applications at any scale.

01

Language-Level Support

Core concurrency primitives within the language itself, including the `synchronized` keyword and the `volatile` modifier.

02

Core Libraries

The Java standard library ships with essential threading utilities — `Thread`, `Runnable`, and `Object` wait/notify mechanisms.

03

High-Level APIs

Java 5+ introduced `java.util.concurrent`, abstracting away low-level complexity for modern, maintainable concurrent code.

The Building Blocks — Threads and Processes

Before diving into high-level APIs, it is essential to understand the fundamental units of concurrent execution: processes and threads — and how Java leverages them.

Process and Thread

Basic units of execution

In concurrent programming, there are two basic units of execution: *processes* and *threads*. In the Java programming language, concurrent programming is mostly concerned with threads

Multithreaded execution is an essential feature of the Java platform. Every application has at least one thread. From the application programmer's point of view, you start with just one thread, called the *main thread*. This thread has the ability to create additional threads

Processes vs. Threads: A Tale of Two Executions

Processes

Isolated, self-contained instances of running programmes — for example, the JVM itself. Each process owns its own memory space and system resources, making cross-process communication costly and complex.

- Isolated memory space
- High creation overhead
- Strong fault isolation

Threads

Lightweight execution units that live *within* a process. Threads share the same heap memory and resources, making them faster to create and more efficient — but also more prone to concurrency hazards.

- Shared memory space
- Low creation overhead
- Java's primary concurrency unit

Defining and Starting a Thread: Bringing Code to Life

Java offers two primary approaches to defining concurrent tasks. The `Runnable` interface is generally preferred as it separates task logic from thread mechanics and works seamlessly with modern executor frameworks.

1

Extend the `Thread` Class

Override the `run()` method and call `start()` on your subclass. Simple, but limits flexibility as Java does not support multiple inheritance.

2

Implement `Runnable`

Define a class implementing `Runnable`, pass it to a `Thread` constructor, and call `start()`. Preferred — decouples task logic from thread management.

3

Invoke `start()`

Always call `start()`, never `run()` directly. Calling `run()` executes code on the *current* thread — no new thread is created.

Extend the `Thread` Class

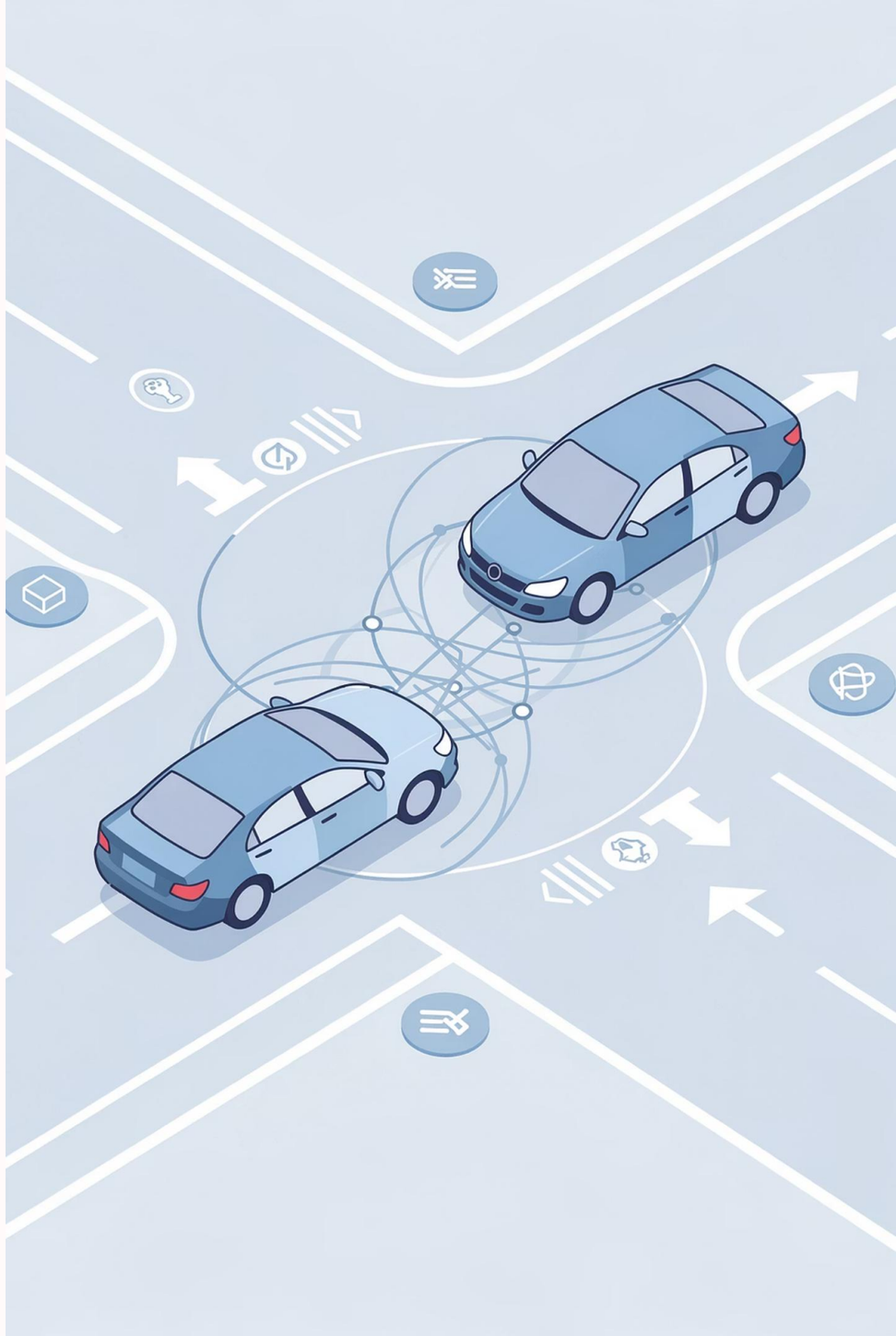
```
public class HelloThread extends Thread {


public void run() {
        System.out.println("Hello from a thread!");
}


public static void main(String args[]) {
        (new HelloThread()).start();
}


}
```

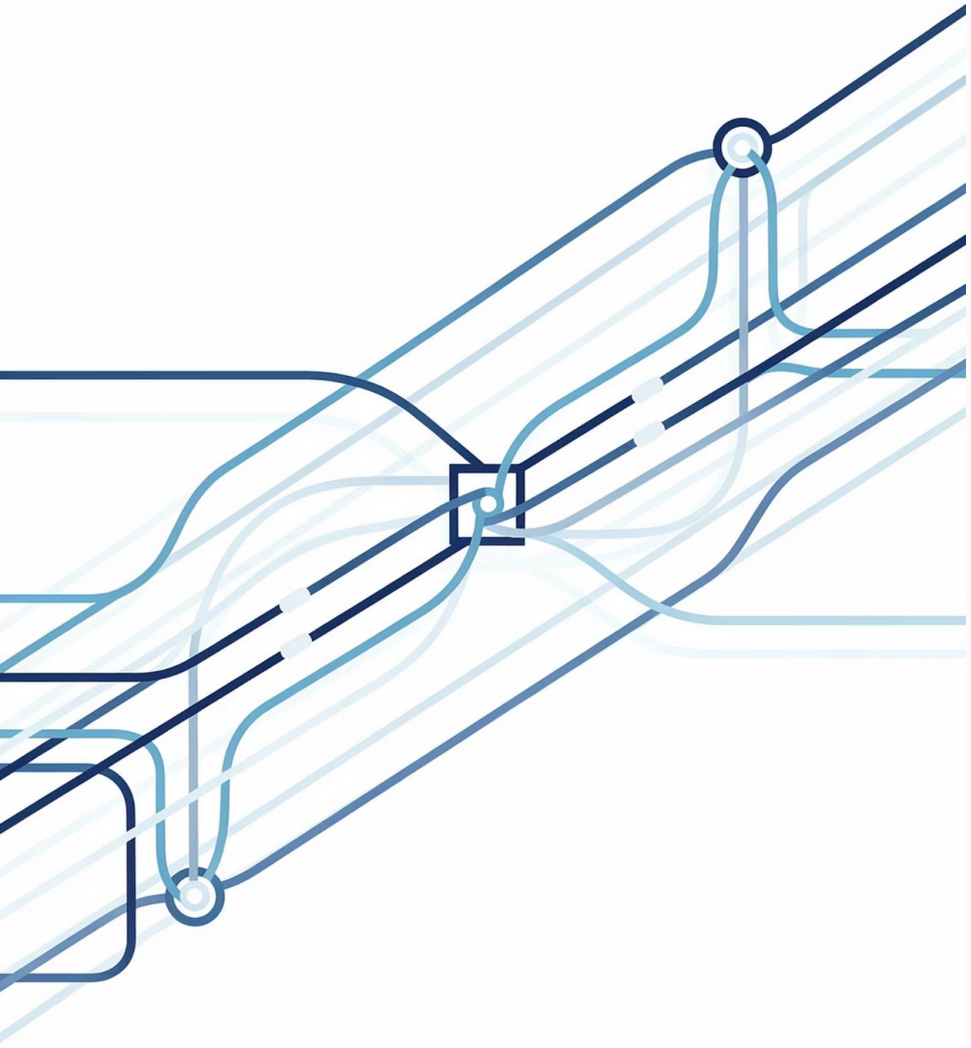
Implement `Runnable`

```
public class HelloRunnable implements Runnable {

public void run() {
        System.out.println("Hello from a thread!");
}


public static void main(String args[]) {
        (new Thread(new HelloRunnable())).start();
}


}
```

The Pitfalls of Parallelism — Synchronisation Challenges

Concurrency introduces a class of subtle, hard-to-reproduce bugs. Understanding thread interference, memory consistency errors, and synchronisation is critical to writing correct concurrent programmes.





Thread Safety

Understanding the foundations of writing correct, reliable concurrent Java applications

Understanding Thread Safety

Correctness First

Thread safety ensures programs behave correctly when multiple threads access shared data simultaneously

The Risks

Unsynchronised access to shared data can lead to race conditions, corrupted state, and unpredictable behaviour

Core Goals

Balance correctness, performance, and maintainability in concurrent applications

What is Thread Safety?

Definition

A class is thread-safe when it behaves correctly when accessed from multiple threads, regardless of scheduling or interleaving of execution

Key Characteristics

- Consistency: Thread-safe classes guarantee consistent results despite concurrency challenges
- Correctness: No data races or corrupted state, even under heavy concurrent load
- Reliability: Predictable behaviour across different execution scenarios

Non-thread-safe classes risk subtle bugs that manifest only under specific timing conditions, making them particularly difficult to debug.

Thread Interference

Thread interference occurs when two or more threads access and modify shared data concurrently without coordination. The result is a **race condition** — where the outcome depends unpredictably on thread scheduling.

Classic Example: Two threads each increment a shared counter. The operation `counter++` is *not* atomic — it involves a read, an increment, and a write. If both threads read the same value before either writes back, one increment is silently lost.

Key Risk: These bugs are non-deterministic and may only manifest under high load or specific timing conditions, making them extremely difficult to diagnose.

Thread A

Reads counter = 5 → increments to 6 → writes 6

Thread B

Reads counter = 5 (stale!) → increments to 6 → writes 6

Result

Expected: 7 — Actual: 6. One increment is silently lost.

The Problem: Race Conditions

Race conditions represent one of the most insidious bugs in concurrent programming. They occur when the correctness of a computation depends on the relative timing of multiple threads.

What Happens

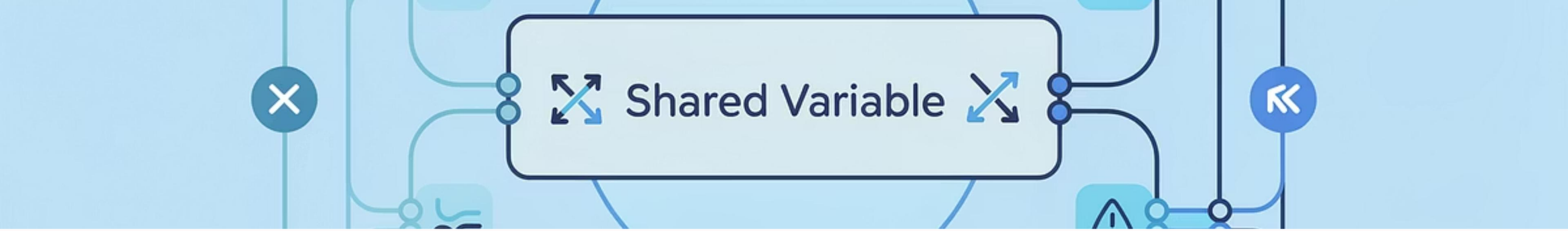
Two or more threads access shared data without proper synchronisation, leading to a "race" to read or modify values

Classic Example

Incrementing a shared counter without atomicity: Thread A reads value 5, Thread B reads value 5, both increment to 6, but the correct result should be 7

The Consequence

Unpredictable and incorrect outcomes that are extremely difficult to reproduce and debug in production environments



Visualising Race Conditions

When two threads simultaneously attempt to update a shared variable, the lack of synchronisation causes operations to interleave in ways that produce incorrect final values.

1. Thread A Reads — Value = 5
2. Thread B Reads — Value = 5
3. Thread A Writes — Value = 6
4. Thread B Writes — Value = 6

Expected result: 7 | Actual result: 6 — One increment was lost due to the race condition

Memory Consistency Errors

Even without direct interference, threads can operate on stale data. Modern CPUs cache values in registers and L1/L2 caches, while compilers reorder instructions for performance. The result: one thread may not see updates made by another.

CPU Caching

Each CPU core may hold a locally cached copy of a variable. Without proper synchronisation, changes made on one core are invisible to threads running on another.

Compiler Reordering

The Java compiler and JIT engine may reorder instructions for performance, causing operations to appear out-of-sequence from another thread's perspective.

The `volatile` Keyword

Declaring a variable `volatile` ensures that reads and writes go directly to main memory, establishing a *happens-before* relationship — but it does not make compound operations atomic.

Synchronisation: The Key to Order

Synchronisation is Java's primary mechanism for preventing thread interference and memory consistency errors. By marking methods or code blocks with the `synchronized` keyword, only one thread at a time can execute that critical section.

Synchronisation establishes two guarantees: **mutual exclusion** (only one thread holds the lock at a time) and a **happens-before** relationship (changes made before releasing the lock are visible to the next thread that acquires it).

Trade-off: Excessive synchronisation causes **thread contention** — threads blocking each other and waiting for locks — which can negate the performance benefits of concurrency entirely.

Synchronisation Guarantees

Mutual Exclusion

Only one thread executes the critical section at any given time.

Visibility

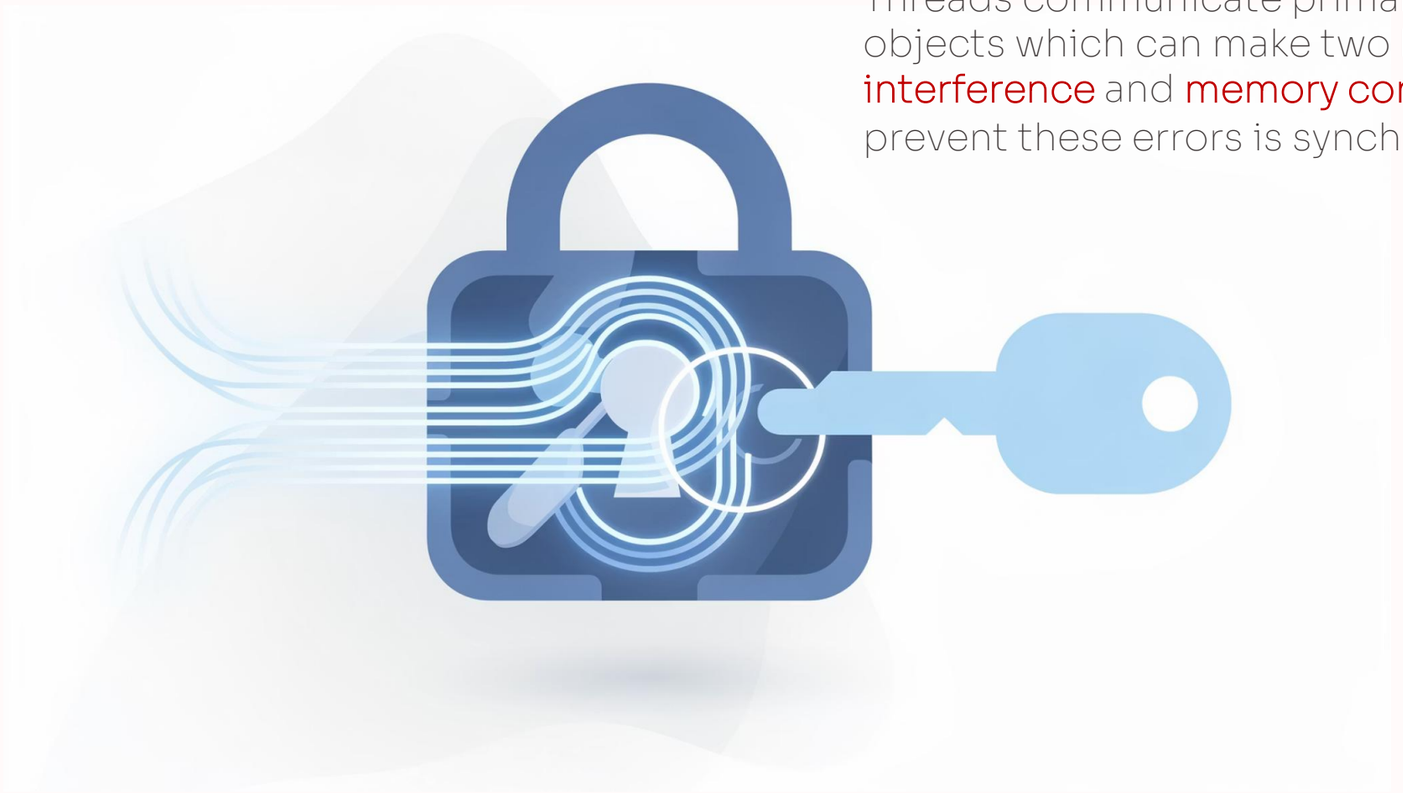
Changes are flushed to main memory and visible to all subsequent threads.

Ordering

Establishes a happens-before relationship between threads.

SYNCHRONISATION

Synchronisation: The Core Mechanism



Threads communicate primarily by sharing access to fields and the objects which can make two kinds of errors possible: thread interference and memory consistency errors. The tool needed to prevent these errors is synchronization.

Synchronisation is Java's fundamental mechanism for coordinating access to shared data. It provides two critical guarantees that are essential for thread safety.

Synchronisation uses intrinsic locks (monitors) that are associated with every Java object, enforcing exclusive access to critical sections of code.

Mutual Exclusion

Only one thread can execute a synchronised block at a time, preventing simultaneous access

Memory Visibility

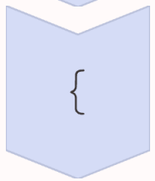
Changes made by one thread become visible to other threads, ensuring consistent state

The `synchronized` Keyword

Java's built-in synchronisation mechanism comes in two primary forms, each serving different use cases and offering different granularity of control.

Method-Level Synchronisation

Locks the entire method using the object's intrinsic lock. Simple but less flexible — the entire method becomes a critical section



Block-Level Synchronisation

Locks only a specific code block, allowing finer-grained control. You can choose which object's lock to acquire

```
// Method-level
public synchronized void increment() {
        count++;
}

// Block-level
public void increment() {
 synchronized(this) {
 count++;
 }
}
```

Locks Beyond `synchronized`

Explicit Lock Objects

The `java.util.concurrent.locks.Lock` interface provides more sophisticated locking mechanisms than the built-in synchronised keyword.

ReentrantLock is the most commonly used implementation, offering advanced features that give developers greater control over locking behaviour.

tryLock()

Attempt to acquire lock without blocking indefinitely

Timed Waits

Specify maximum wait time for lock acquisition

Interruptible

Lock acquisition can respond to thread interruption

Fair Queuing

Optional guarantee that longest-waiting thread acquires lock first

Immutable Objects: Thread Safety by Design

Immutability is the most elegant approach to thread safety. Objects that cannot change state after construction are inherently thread-safe without any synchronisation overhead.

Perfect Thread Safety

Immutable objects can be safely shared between threads without synchronisation because their state never changes

Zero Synchronisation Cost

No locks, no contention, no performance overhead — immutable objects are always safe to access concurrently

Java Examples

`String`, `Integer`, and all wrapper classes are immutable. `BigInteger` and `BigDecimal` are thread-safe through immutability

```
public class Person {

    private String name;
    private int age;

    // Constructor
    public Person(String name, int age){
        synchronized (this) {
        this.name = name;
        this.age = age;
      }

    }

    // Getters only (no setters)
    public synchronized String getName(){
        return name;
    }

    public synchronized int getAge() {
        return age;
    }
}
```

```
Person p1 =
    new Person("Mohit", 25);
...
int myAge = p1.getAge();       //Statement 1
String myName = p1.getName(); //Statement 2
```

If another thread invokes `p1.set` after Statement 1 but before Statement 2, the value of `myName` won't match the value of `myAge`. To avoid this outcome, the two statements must be bound together:

```
synchronized (p1) {
    int myAge = p1.getRGB();
    String myName = p1.getName();
}
```

This kind of inconsistency is only possible for mutable objects — it will not be an issue for the immutable version of Person.

```
final class Person {

      private final String name;
      private final int age;
...
...
```

VOLATILE

Volatile Variables: Visibility Without Mutual Exclusion

How Volatile Works

The volatile keyword ensures that changes to a variable are immediately visible to all threads. Reads always see the most recent write, without caching or optimisation delays.

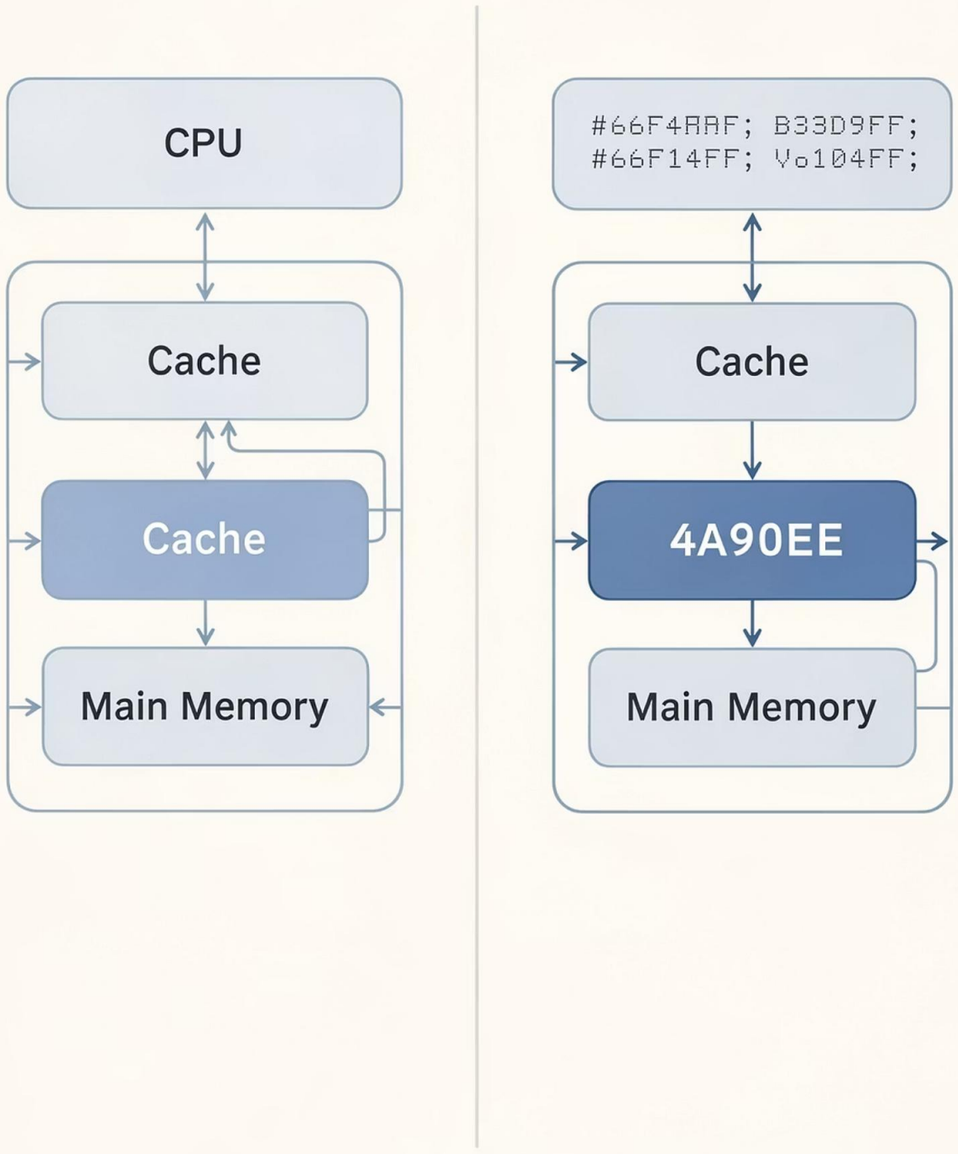
Appropriate Use Cases

- Boolean flags for thread termination
- State indicators and status variables
- Simple publish-subscribe patterns

Important Limitation

Volatile does not provide atomicity for compound actions. Operations like increment (read-modify-write) still require synchronisation or atomic variables.

```
private volatile boolean stopRequested;

public void shutdown() {
 stopRequested = true;
}
```



Memory Visibility: With and Without Volatile

Without Volatile

Threads may cache variables locally. Changes made by one thread might not be visible to others, leading to stale reads and inconsistent state.

With Volatile

All reads and writes go directly to main memory. Every thread sees the most up-to-date value, ensuring consistent visibility across the entire application.

Atomic Variables: Lock-Free Thread Safety

The java.util.concurrent.atomic package provides powerful lock-free alternatives for common thread-safety scenarios, combining performance with correctness.

Atomic Classes Available

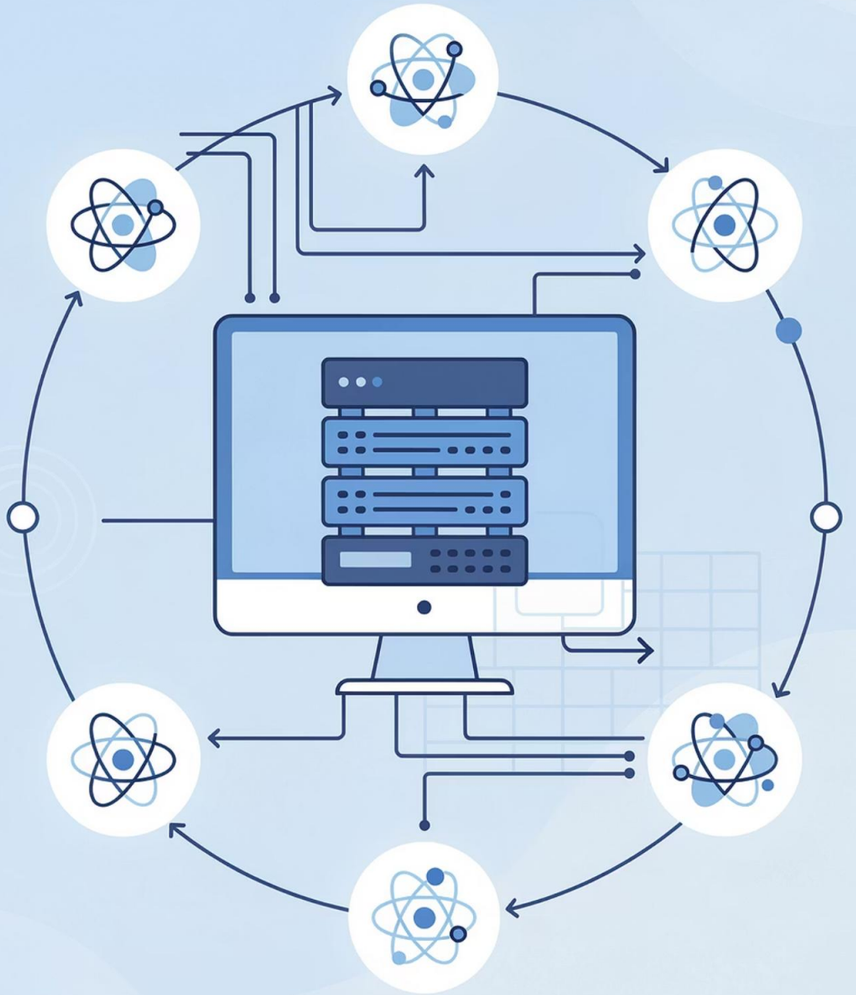
AtomicInteger, AtomicLong, AtomicBoolean, and AtomicReference provide atomic operations on their respective types without explicit locking.

Supported Operations

Support atomic read-modify-write operations like incrementAndGet(), compareAndSet(), and getAndSet() using efficient hardware-level instructions.

Ideal Use Cases

Perfect for counters, sequence generators, state flags, and reference management where high contention might make locks expensive.



Example: Thread-Safe Counter Implementations

Naïve Counter (Broken)

```
private int count;

public void increment() {
   count++; // RACE CONDITION!
}

public int get() {
   return count; // MAY BE STALE!
}
```

No synchronisation leads to lost updates and visibility problems. Multiple threads can read the same value and overwrite each other's increments.

Synchronized Counter

```
private int count;

public synchronized void increment() {
   count++;
}

public synchronized int get() {
   return count;
}
```

Synchronized methods ensure atomicity and visibility. Only one thread can execute increment() or get() at a time, preventing race conditions.

Atomic Counter (Best)

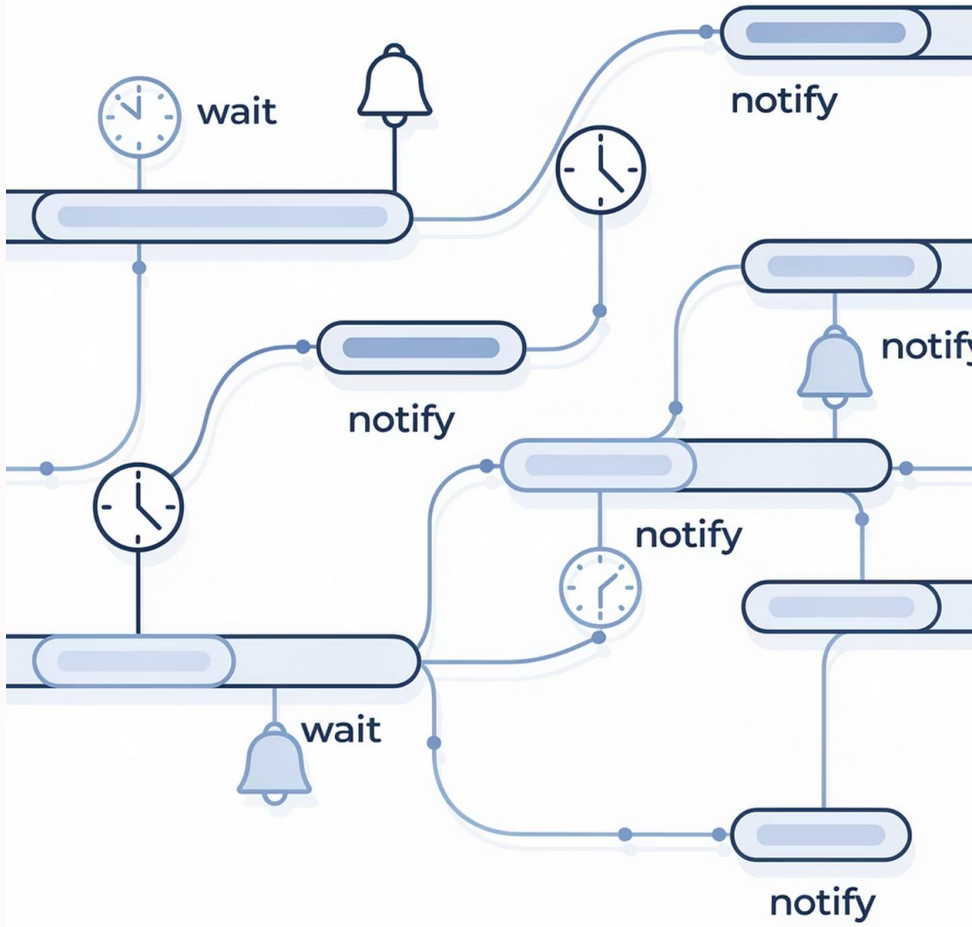
```
private AtomicInteger count =
   new AtomicInteger(0);

public void increment() {
   count.incrementAndGet();
}

public int get() {
   return count.get();
}
```

AtomicInteger provides lock-free thread safety using efficient hardware instructions, offering better performance under high contention.

COORDINATION

Coordination Between Threads: Wait and Notify

Sharing objects frequently requires threads to coordinate their actions, waiting for certain conditions before proceeding. Java provides built-in mechanisms for this essential pattern.



1 Wait and Notify Basics

Use Object.wait() to release a lock and wait for a condition, and notify() or notifyAll() to wake waiting threads. These methods must be called inside synchronized blocks.

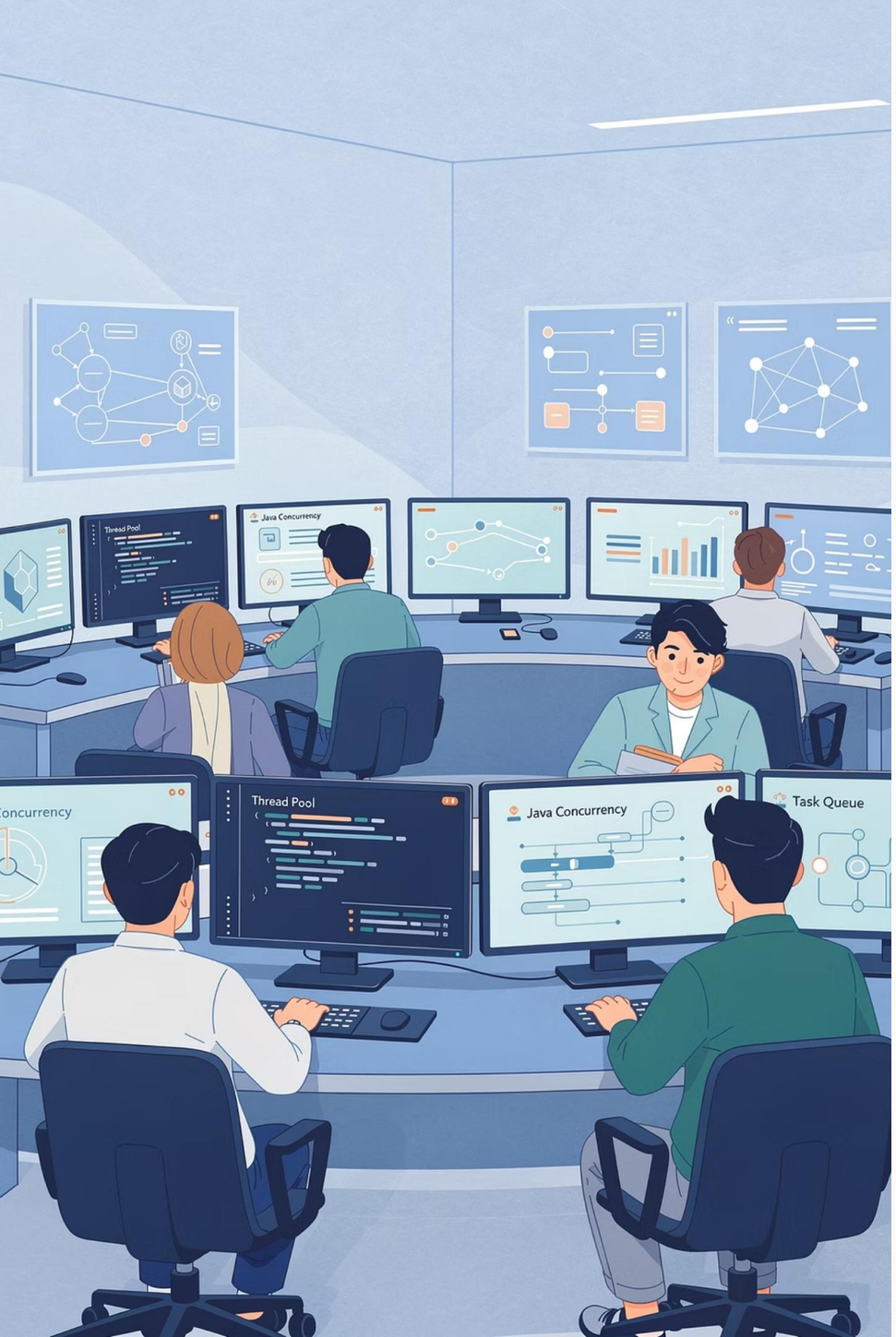
2 Producer-Consumer Pattern

Classic example involves a shared buffer where producers add items and consumers remove them. Coordination ensures producers wait when full and consumers wait when empty.

```
public void guardedJoy() {
    // Simple loop guard. Wastes
    // processor time. Don't do this!
    while(!joy) {}
    System.out.println("Joy has been achieved!");
}
```

```
public synchronized void guardedJoy() {
    // This guard only loops once for each special
event, which may not
 // be the event we're waiting for.
    while(!joy) {
        try {
            wait();
        } catch (InterruptedException e) {}
    }
    System.out.println("Joy and efficiency have been
achieved!");
}
```

```
public synchronized notifyJoy() {
    joy = true;
    notifyAll();
}
```



High-Level Concurrency Objects — Powering Modern Applications

Java 5 introduced `java.util.concurrent`, a comprehensive toolkit that replaces error-prone manual thread management with robust, well-tested high-level abstractions designed for production-grade systems.

The `java.util.concurrent` Package: A Toolkit for Concurrency

Introduced in Java 5.0, `java.util.concurrent` was a landmark addition to the Java platform. Designed by concurrency experts including Doug Lea, it provides battle-hardened building blocks that dramatically raise the level of abstraction for concurrent programming.

Less Boilerplate

Significantly reduces the need for manual, error-prone low-level synchronisation using `wait()`, `notify()`, and `synchronized` blocks.

Better Performance

Implementations are highly optimised for the JVM, often outperforming hand-written synchronisation through lock-free algorithms and fine-grained locking.

Proven Reliability

Extensively tested, peer-reviewed implementations that handle edge cases developers often miss when rolling their own concurrency utilities.

Maintainability

High-level abstractions produce cleaner, more readable code that is easier to reason about, review, and maintain over time.

Executors: Managing Thread Lifecycles

The Executor framework separates task submission from thread management. Rather than creating threads manually, you submit tasks to an executor and let it handle pooling, queuing, and lifecycle management — resulting in far more predictable resource consumption.

`Executor` Interface

The simplest abstraction — a single `execute(Runnable)` method. Decouples task submission from execution mechanics entirely.

`ExecutorService`

Extends `Executor` with thread pool management, task queuing, `Future`-based result retrieval, and graceful shutdown control.

`ScheduledExecutorService`

Supports delayed and periodic task execution — ideal for polling, timeouts, and recurring background jobs.

`Executors` Factory

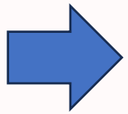
Provides convenient factory methods: `newFixedThreadPool()`, `newCachedThreadPool()`, `newSingleThreadExecutor()`, and more.

Executors: Managing Thread Lifecycles

`Executor` Interface 

```
public interface Executor {
    void execute(Runnable command);
}
```

Base interface
Method: execute(Runnable)
Executes tasks
No thread pooling



Even though we are using Executor,
we are still creating threads manually inside it. This is a basic implementation, not efficient like thread pools

```
import java.util.concurrent.Executor;

class SimpleExecutor implements Executor {
        public void execute(Runnable task) {
        new Thread(task).start(); // manual
thread creation
}
}

public class ExecutorDemo {
        public static void main(String[]
args) {
        Executor executor = new
SimpleExecutor();

        executor.execute(() -> {
System.out.println("Task executed by
Executor");
});
}
}
```

Executors: Managing Thread Lifecycles

`ExecutorService` 

Extends Executor
Supports thread pool
Methods: submit(), shutdown()
Manages lifecycle

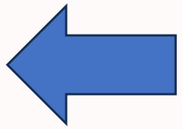
```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ExecutorServiceDemo {

    public static void main(String[] args) {

        ExecutorService executor =
Executors.newFixedThreadPool(2);

        executor.submit(() -> {
            System.out.println("Task 1 running");
        });

        executor.submit(() -> {
            System.out.println("Task 2 running");
        });

        executor.shutdown(); // important
    }
}
```

Executors: Managing Thread Lifecycles

`ScheduledExecutorService`

Extends ExecutorService
Supports delayed & periodic tasks
schedule(), scheduleAtFixedRate()

```
Time 0 sec → "Repeating task"
Time 2 sec → "Repeating task"
Time 3 sec → "Task executed after delay"
Time 4 sec → "Repeating task"
Time 6 sec → "Repeating task"
...
```



```java
import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.TimeUnit;

public class ScheduledExecutorDemo {

    public static void main(String[] args) {

        ScheduledExecutorService scheduler =
                Executors.newScheduledThreadPool(1);

        // Run after 3 seconds
        scheduler.schedule(() -> {
            System.out.println("Task executed after
delay");
        }, 3, TimeUnit.SECONDS);

        // Run every 2 seconds
        scheduler.scheduleAtFixedRate(() -> {
            System.out.println("Repeating task");
        }, 0, 2, TimeUnit.SECONDS);
    }
}
```

Executors: Managing Thread Lifecycles

`Executors` Factory 

- A utility class that provides ready-made thread pools
- newFixedThreadPool()
- newCachedThreadPool()
- newSingleThreadExecutor()

- Fixed: fixed threads
- Cached: dynamic threads
- Single: one thread
- Scheduled: timed tasks

```
ExecutorService executor =
Executors.newFixedThreadPool(3);

ExecutorService executor =
Executors.newCachedThreadPool();

ExecutorService executor =
Executors.newSingleThreadExecutor();
```

Concurrent Collections: Thread-Safe Data Structures

Thread-safe data structures from `java.util.concurrent` allow multiple threads to access and modify data safely without explicit synchronization.

`CopyOnWriteArrayList`

Creates a fresh copy on every write. Perfect for read-heavy workloads where iteration must never block.

BlockingQueue

additionally supports operations that wait for the queue to become non-empty when retrieving an element,, wait for space to become available in the queue when storing an element.

```
import java.util.concurrent.CopyOnWriteArrayList;

public class CopyOnWriteDemo {

    public static void main(String[] args) {

        CopyOnWriteArrayList<String> list = new
CopyOnWriteArrayList<>();

        list.add("A");
        list.add("B");

        // Thread 1: Reading
        new Thread(() -> {
            for (String item : list) {
                System.out.println("Reading: " + item);
                try { Thread.sleep(100); } catch(Exception e){}
            }
        }).start();

        // Thread 2: Writing
        new Thread(() -> {
            list.add("C");
            System.out.println("Added C");
        }).start();
    }
}
```

Lock Objects & Atomic Variables: Finer-Grained Control

Lock Objects - Advanced synchronization tools from java.util.concurrent.locks that give explicit control over locking..

Lock Objects — `ReentrantLock`

Offers all the capabilities of `synchronized` plus: try-lock (non-blocking attempt), timed lock acquisition, and interruptible locking., also supports fair queuing.

```
import java.util.concurrent.locks.ReentrantLock;

class Example {
    ReentrantLock lock = new ReentrantLock();

    void outer() {
        lock.lock();
        try {
            System.out.println("Outer method");
            inner();  // calling another method that also uses lock
        } finally {
            lock.unlock();
        }
    }

    void inner() {
        lock.lock();  // same thread locking again
        try {
            System.out.println("Inner method");
        } finally {
            lock.unlock();
        }
    }
}
```

Lock Objects & Atomic Variables: Finer-Grained Control

Atomic Variables — `AtomicInteger`

They allow thread-safe operations without using locks

Classes like `AtomicInteger`, `AtomicLong`, and `AtomicReference` provide lock-free, thread-safe operations using CPU-level compare-and-swap (CAS) instructions — delivering excellent performance for simple counters and flags.

The Well-Stocked Toolbox: `java.util.concurrent`

Think of `java.util.concurrent` as a master craftsman's toolbox. Rather than fashioning a crude tool by hand each time, you reach for a precision instrument purpose-built for the job — Executors to manage thread lifecycles, Concurrent Collections for safe data sharing, Locks for fine-grained coordination, and Atomics for lock-free operations. The right tool, every time.

Common Misconceptions

Myth: "Concurrency is just about threads"

Reality: It's fundamentally about **coordination** and **state management**. Threads are just the mechanism; the real challenge is safely managing shared mutable state across those threads.

Myth: "Synchronisation solves everything"

Reality: Synchronisation is not a silver bullet. Overuse degrades performance, whilst improper use can introduce **deadlocks** and **contention**—sometimes making problems worse.

Summary

01

Concurrency is Unavoidable

In modern software development, concurrent programming is no longer optional. Multi-core processors and user expectations demand it.

02

Understanding is Essential

Mastering concurrency's pitfalls and tools is critical for building robust, production-ready Java applications that users can rely on.

> "The art of concurrent programming lies not in avoiding complexity, but in managing it systematically."